



Mansoura University
Faculty of Computers and Information
Department of Computer Science



BookOrbit: A Peer-to-Peer Book Lending Platform for University Students

Project Proposal

Submitted by

- | | | |
|--|------------|---------|
| 1. ادهم محمود احمد الطنطاوي | 1000324542 | Sec: 8 |
| 2. اياد رضا عبدالمؤمن عبدالفتاح محمد | 1000338636 | Sec: 11 |
| 3. عبدالرحمن محي الدين محمود محمد عبدالمعطي ابوالعطا | 1000324059 | Sec: 23 |
| 4. عبدالرحمن ياسر هلال هلال الالفي | 1000339823 | Sec: 23 |
| 5. عمر اشرف عبدالستار محمد الازمازي | 1000339821 | Sec: 25 |
| 6. عبدالرحمن محمد حمدي سعد محمد نوفل | 1000324812 | Sec: 22 |

Supervisors:

TA: Amany Sherif

INTRODUCTION:

In universities across Egypt and many developing countries, the cost of books is a recurring challenge that often goes unaddressed. Students are not only expected to keep up with academic textbooks and course references, but many also want access to a wider range of reading materials, such as novels, self-improvement books, and general non-fiction, and the cumulative expense across all of these can quickly become a real burden, particularly for those from modest financial backgrounds. University libraries help to some extent, but they typically hold only a handful of copies of popular titles and rarely stock non-academic reading, which means long waiting lists and dry spells right when students need those books the most.

The frustrating part is that a solution is already sitting right there within the student community. Countless books of all kinds are gathering dust on shelves, no longer needed by their owners but potentially useful to someone else in the same university. The problem is that there is no proper channel for this exchange. Book sharing does happen, but only informally through social media posts, word-of-mouth, or chance hallway conversations. These methods offer no accountability, no way to track a book's condition, and no real guarantee that a borrowed book will ever come back.

BookOrbit is a web-based platform that addresses this issue by enabling a peer-to-peer book lending system within the university community. The system allows students to list books they own, browse available copies, and send borrowing requests, while administrators manage users and the catalog. To ensure accountability and encourage active participation, BookOrbit introduces a points-based economy: students earn points by lending their books and receiving positive reviews, and they spend points when borrowing from others. Every new student begins with a small initial balance to lower the barrier to entry, and the system automatically computes point transactions based on predefined business rules.

The application is built using a modern full-stack architecture, with a React-based frontend and an ASP.NET Core 9 backend following Clean Architecture and SOLID principles. Entity Framework Core is used to manage a SQL Server database that ensures persistent and structured data storage. The system models core entities such as Student, Book, BookCopy, LendingListRecord, BorrowingRequest, BorrowingTransaction,

BorrowingReview, and PointTransaction, with clearly defined relationships between them. Security is implemented using JWT authentication with refresh token rotation, and system observability is supported through OpenTelemetry for monitoring and performance tracking. Access to the platform is restricted to verified Mansoura University emails (ending in @std.mans.edu.eg), and new student accounts must pass through a multi-step administrative approval process before gaining full access.

PROJECT OBJECTIVES:

The central objective of BookOrbit is to design and implement a secure, well-architected web application that enables peer-to-peer book lending among university students in a structured and accountable manner. This broad goal can be broken down into the following specific objectives:

1. **Build a functional peer-to-peer book lending platform** that allows Mansoura University students to register physical book copies they own, browse listings posted by fellow students, submit borrowing requests, and manage the complete lending lifecycle from listing to return and review.
2. **Implement a self-sustaining points economy** that rewards lending behavior and responsible borrowing. Points serve as the platform's internal currency, automatically awarded for lending books and receiving favorable reviews, and deducted upon borrowing. This gamification layer is designed to maintain an active and balanced user base without requiring monetary transactions.
3. **Enforce community trust** through university email verification (restricting registration to @std.mans.edu.eg addresses) and a multi-step administrative approval process. Students pass through a defined state lifecycle (Pending → Approved → Active) before gaining full access to the platform's features.
4. **Apply Clean Architecture and Domain-Driven Design** to structure the backend system, ensuring that business logic resides exclusively in the Domain and Application layers while infrastructure concerns (database access, email services, caching) remain isolated behind well-defined interfaces. This separation enables independent testing, straightforward maintenance, and technology flexibility.

5. **Implement the CQRS pattern using MediatR**, providing a clean and scalable approach to handling all read and write operations. Every use case in the system is represented as either a Command (for write operations) or a Query (for read operations), each with its own handler, validator, and data transfer object.
6. **Design entity state machines** that govern the lifecycle of key domain objects (Students, Books, BookCopies, Lending Records, Borrowing Requests, and Transactions), preventing invalid state transitions through guarded methods that return typed Result objects instead of throwing exceptions.
7. **Implement robust security** including JWT-based authentication with refresh token rotation, email verification, password recovery, and fine-grained policy-based authorization using ASP.NET Core's built-in authorization framework with seven custom policies.
8. **Integrate system observability** using OpenTelemetry, exporting distributed traces, application metrics, and structured logs to industry-standard tools (Jaeger, Prometheus, Grafana, and Seq) for real-time monitoring and diagnostics.
9. **Containerize the entire application** using Docker and Docker Compose, enabling consistent development, testing, and deployment environments that can be reproduced with a single command.
10. **Integrate a fully functioning Email Service** to provide reliable, secure, and timely communication with users. This component actively notifies users about important events, including account registration confirmation, OTP delivery, password reset requests, borrowing request updates, and reminders for due dates. It utilizes retry mechanisms for high delivery reliability and ensures all communications are sent only to authenticated, verified university emails.

PROJECT SIGNIFICANCE:

BookOrbit is not just a software project. It addresses a real problem students face every semester while also putting advanced software engineering concepts into actual practice.

From a social and economic perspective, the platform directly tackles the financial burden that textbooks impose on students. By enabling peer-to-peer lending rather than purchasing, students can access the materials they need at no monetary cost, exchanging only points earned through their own participation. This creates a circular economy of knowledge sharing where every participant both contributes to and benefits from the community. The platform also creates opportunities for students who might not otherwise interact to connect with each other, which helps build a sense of community within the university.

On the technical side, BookOrbit is a hands-on exercise in modern software engineering. The project applies Clean Architecture, Domain-Driven Design, and the CQRS pattern in a real working context, showing how these industry-standard practices produce maintainable, testable, and scalable software. The use of state machines for entity lifecycle management, the Result pattern for explicit error handling, and Value Objects for domain-level validation together form a defensive programming approach that catches entire categories of bugs at the design level, rather than relying on runtime checks. These same patterns are widely used in production systems at large software companies to build systems that are meant to last.

The security implementation covers the critical areas every modern web application must handle: JWT authentication with token rotation, seven custom authorization policies, and mandatory email verification. Similarly, the integration of OpenTelemetry for distributed tracing, metrics collection, and structured logging reflects a level of operational readiness that goes well beyond what most academic projects attempt.

Finally, containerizing the whole stack with Docker follows the same DevOps approach used in professional environments. The entire setup, including the API server, SQL Server database, and monitoring infrastructure, can be brought up with a single `docker-compose up` command. This gives the team hands-on experience with container orchestration and infrastructure-as-code, both of which are practical skills that carry real value in the industry.

HW AND SW RESOURCES:

Software Tools	C# (.NET 9), ASP.NET Core 9 Web API, Entity Framework Core 9, MediatR (CQRS & Mediator Pattern), FluentValidation, ASP.NET Core Identity, HybridCache (.NET 9), OpenTelemetry SDK, React 19 with TypeScript.
Software	JetBrains Rider / Visual Studio 2022 / VS Code, SQL Server 2022, Docker Desktop & Docker Compose, Postman (API Testing), Git & GitHub, Seq (Log Management), Jaeger (Distributed Tracing), Prometheus & Grafana (Metrics & Dashboards).
Hardware	Standard development workstations. No specialized hardware is required; the application runs on any machine capable of running Docker containers.

RELATED WORKS:

A number of platforms and research papers have explored book sharing, library management, and community exchange systems. BookOrbit takes some inspiration from these, but brings its own angle by combining peer-to-peer lending with a gamified points economy, strict community verification, and a carefully structured software architecture, all designed around the specific needs of a university environment.

BookMooch [1] is an online book exchange platform that allows users worldwide to give and receive books using a credit-based system. Users earn points by mailing books to others and spend points to request books. While BookMooch's credit system inspired BookOrbit's points economy, it operates on a global scale with physical mail delivery, making it impractical for localized, same-campus exchanges where students can meet in person. BookOrbit narrows the scope to a single university community, which simplifies logistics and builds stronger trust.

PaperBackSwap [2] follows a similar model, enabling members to swap paperback books across the United States. It uses a credit system and maintains a wish list feature. However, like BookMooch, it relies on postal shipping and lacks the closed-community trust model that BookOrbit provides through university email verification and administrative oversight.

Little Free Library [3] is a global nonprofit that establishes physical book-sharing boxes in public spaces, operating on an honor system where anyone can take or leave a book. While the spirit of community book sharing aligns with BookOrbit's mission, the lack of any digital tracking, accountability, or user management makes it unsuitable as a model for a university-scale system where book return guarantees are important.

In the academic literature, Chowdhury and Gibb [4] examined the role of digital library services in supporting education and found that access to learning resources remains one of the primary challenges facing students in developing countries. Their work highlights the need for low-cost, community-driven solutions that supplement institutional libraries, which is precisely the gap BookOrbit aims to fill.

On the gamification front, Hamari et al. [5] conducted a comprehensive review of gamification in educational contexts and found that point-based reward systems significantly increase user engagement and participation when the rewards are directly tied to meaningful actions. BookOrbit applies this principle by linking point earnings to concrete lending activities and peer reviews, rather than arbitrary achievements.

From a software engineering perspective, Robert C. Martin's *Clean Architecture* [6] and Eric Evans' *Domain-Driven Design* [7] provide the theoretical foundation for BookOrbit's backend design. While many academic projects use simpler architectural patterns (such as the traditional three-tier architecture), BookOrbit deliberately adopts Clean Architecture with strict dependency inversion to demonstrate how production-grade enterprise patterns can be applied in a university project context.

The CQRS pattern, as described by Greg Young [8], separates read and write concerns into distinct models. BookOrbit implements this using the MediatR library, which provides a mediator-based request/response pipeline with cross-cutting behaviors for validation, caching, performance monitoring, and exception handling.

PROPOSED METHODOLOGY (OUR SOLUTION):

Building BookOrbit involved more than picking a framework and writing code. We combined several architectural patterns and development practices, each one addressing a specific concern in the system. The subsections below walk through the main decisions we made and the reasoning behind each of them.

1. Clean Architecture with Strict Layer Separation

The backend is structured into four distinct layers, each with clearly defined responsibilities and dependency rules. The innermost layer, **BookOrbit.Domain**, contains all business entities, value objects, domain events, and state machine logic. It has zero external dependencies: no references to frameworks, databases, or third-party libraries. The **BookOrbit.Application** layer defines use cases as Commands and Queries, along with their handlers, validators, and data transfer objects. It depends only on the Domain layer and defines interfaces (ports) for any infrastructure service it needs. The **BookOrbit.Infrastructure** layer implements those interfaces, providing concrete implementations for database access (via Entity Framework Core), email services, caching, authentication, and file storage. Finally, the **BookOrbit.Api** layer serves as the entry point, hosting ASP.NET Core controllers, middleware, and configuration. Dependencies always point inward: the Api layer depends on Infrastructure and Application, Infrastructure depends on Application and Domain, Application depends only on Domain, and Domain depends on nothing.

2. Domain-Driven Design (DDD)

Core business concepts are modeled as rich domain entities with encapsulated behavior rather than anemic data containers. Each entity enforces its own invariants through private setters, factory methods, and guarded state transitions. For example, a **Student** entity cannot transition from **Pending** to **Active** without first passing through **Approved**, and this rule is enforced within the entity itself. Value Objects such as **UniversityMail**, **ISBN**, **StudentName**, and **PhoneNumber** encapsulate validation logic at the type level, making it impossible to construct an invalid instance. Domain Events enable loose coupling between

aggregates. When a student is created, a `StudentCreatedEvent` is raised and handled by independent event handlers without the creating code needing to know about downstream effects.

3. CQRS with MediatR Pipeline

Every user action in the system is represented as either a Command (for operations that change state) or a Query (for operations that read data). This separation allows us to optimize each path independently: commands go through full validation and domain logic, while queries can use lightweight DTO projections with `AsNoTracking()` for better read performance. The MediatR library routes each request through a pipeline of cross-cutting behaviors, including an **UnhandledExceptionBehaviour** for global error logging, a **PerformanceBehaviour** that flags slow requests, a **ValidationBehaviour** that runs FluentValidation rules before the handler executes, and a **CachingBehaviour** that caches query results using .NET 9's HybridCache with tag-based invalidation.

4. The Result Pattern for Error Handling

Instead of throwing exceptions for business rule violations (which is both expensive and semantically incorrect), all domain operations return a `Result<T>` type that explicitly represents either success (with a value) or failure (with a list of typed errors). This "railway-oriented" approach forces calling code to handle both outcomes, eliminating the possibility of unhandled business errors and making the control flow explicit and predictable. Implicit conversion operators allow handlers to simply write `return student;` for success or `return StudentErrors.AlreadyActive;` for failure, keeping the code clean and readable.

5. Security: JWT Authentication and Policy-Based Authorization

Authentication is handled through JWT access tokens (short-lived, stored in memory on the client) paired with refresh tokens (long-lived, stored in the database). On every token refresh, the old refresh token is revoked and a new pair is issued, preventing token replay attacks. Authorization is managed through seven custom ASP.NET Core policies: `ActiveUserAccess`, `AdminOnlyAccess`, `RegisteredUserAccess`,

OwnershipAccess, StudentOwnershipAccess, StudentOnlyAccess, and ActiveStudentAccess. These can be composed together to express complex permission rules declaratively on each endpoint.

6. Observability with OpenTelemetry

The system is instrumented with OpenTelemetry to provide full-stack observability across three dimensions. **Distributed traces** are collected from ASP.NET Core, Entity Framework Core, and HttpClient, then exported to Jaeger for visual inspection of request flows. **Application metrics**, including request rates, latencies, and error counts, are scraped by Prometheus and visualized through Grafana dashboards. **Structured logs** enriched with correlation IDs are sent to Seq, enabling powerful filtering and search across log entries tied to specific requests or users.

7. Strict Registration and Profile Approval Workflow

The platform enforces a strict and multi-layered registration process to ensure that only legitimate and verified users gain access to the system. Every step of this workflow is designed to maintain trust, security, and data integrity across the platform.

The process begins with **email verification**, where users must confirm ownership of their university email address, ensuring that every account is tied to a valid academic identity. Once verified, the user profile enters an **administrative review phase**, during which administrators manually evaluate the submitted profile information to confirm its authenticity and compliance with platform policies.

After review, administrators can:

- **Approve the profile**, granting full access to the platform
- **Reject the profile**, preventing access until any issues are resolved

Beyond initial approval, the system is powered by a **flexible state machine** that allows dynamic control over user statuses. This enables administrators to:

- Temporarily **ban users** in case of violations
- **Unban and restore access** when appropriate

- Transition users between different states in a controlled and traceable manner

This structured lifecycle means user access is governed by clear rules and defined state transitions rather than being granted all at once, which makes the system considerably more secure and easier for administrators to manage.

8. Fully Functioning Email Service

The Email Service handles all user-facing notifications across the platform. It covers a range of important events, including:

- Account registration confirmation
- One-Time Password (OTP) delivery
- Password reset requests
- Borrowing request updates (accepted or rejected)
- Reminders for due dates and overdue returns

The system ensures reliable delivery by implementing retry mechanisms for temporary failures, minimizing the risk of missed notifications. Emails are structured to be clear and easy to act on. The service is also tied directly to the user's verified university email, so all communications go only to authenticated recipients. Rather than being an afterthought, the email service is woven into the platform's core and plays a direct role in security, reliability, and user experience.

9. Containerization with Docker

The entire application stack is containerized using Docker Compose. A single `docker-compose up` command provisions the ASP.NET Core API, SQL Server database, Seq, Jaeger, Prometheus, and Grafana, all pre-configured and connected. This ensures that every team member works in an identical environment, eliminates "works on my machine" issues, and provides a clear path toward production deployment.

Taken together, these choices are not arbitrary. Clean Architecture keeps the codebase maintainable as it grows, DDD keeps the business logic honest and well-contained, CQRS gives us the flexibility to optimize reads and writes separately, and the Result pattern makes error handling explicit rather than hidden in exceptions. On the infrastructure side, JWT

with custom policies handles security, the strict registration workflow builds trust within the community, the email service keeps users informed at every step, OpenTelemetry gives us visibility into what the system is actually doing at runtime, and Docker means the whole thing is reproducible with a single command. The goal throughout was to build something that holds up both as an academic project and as a system that could realistically be put into production.

REFERENCES:

- [1] BookMooch, "BookMooch: a free book exchange." Available: <https://bookmooch.com>
- [2] PaperBackSwap, "PaperBack Swap: Free Online Book Club." Available: <https://www.paperbackswap.com>
- [3] Little Free Library, "About Little Free Library." Available: <https://littlefreelibrary.org>
- [4] S. Chowdhury and F. Gibb, "Digital Library Services in the Developing World: Challenges and Opportunities," *The Electronic Library*, vol. 27, no. 2, pp. 231–243, 2009.
- [5] J. Hamari, J. Koivisto, and H. Sarsa, "Does Gamification Work? A Literature Review of Empirical Studies on Gamification," in *Proc. 47th Hawaii Int. Conf. on System Sciences*, 2014, pp. 3025–3034.
- [6] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [7] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [8] G. Young, "CQRS Documents," 2010. Available: https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf